إنتاج البرمجيات

مقدمة في هندسة البرمجيات :

تُعرَّف هندسة البرمجيات بأنها فرع من فروع الهندسة يُعنى بجميع الجوانب النظرية والعملية المتعلقة بإنتاج البرمجيات، بدءًا من تحليل المتطلبات ووصولًا إلى الصيانة والتطوير المستمر. وتهدف إلى تطبيق مبادئ الهندسة في بناء أنظمة برمجية عالية الكفاءة والجودة، قابلة للصيانة والتطوير، وتلبي احتياجات المستخدمين ضمن حدود الميزانية والجدول الزمني المحددين.

من الناحية التقنية، تعتمد هندسة البرمجيات على مجموعة متكاملة من المنهجيات، النماذج، والأدوات التي تُمكّن من تنظيم عملية التطوير بشكل منهجي ومنضبط، حتى في ظل التغيرات المستمرة في بيئة العمل والتكنولوجيا.

ويُعد هذا التخصص جوهريًا في ضمان موثوقية الأنظمة، وتحقيق التوازن بين جودة البرمجيات وتعقيداتها التقنية والتشغيلية.

مفهوم انتاج البرمجيات :

إنتاج البرمجيات هو العملية الكاملة التي يتم من خلالها تطوير منتج برمجي قابل للاستخدام، ويشمل ذلك جميع المراحل الأساسية مثل التخطيط، التحليل، التصميم، التطوير، الاختبار، والتسليم. ويعد جزءًا أساسيًا من هندسة البرمجيات، حيث توفر الهندسة الإطار المنهجي الذي يُمكّن من إنتاج برامج ذات جودة عالية تلبي احتياجات المستخدمين .

تُعنى **هندسة البرمجيات** بوضع الأسس والمنهجيات المنظمة لتطوير البرمجيات، حيث توفّر إطارًا شاملاً يشمل الأدوات، النماذج، والتقنيات اللازمة لبناء أنظمة عالية الكفاءة والموثوقية.

أما **إنتاج البرمجيات**، فيمثّل الجانب التنفيذي من هذا التخصص، حيث تُطبّق تلك المفاهيم عمليًا لإنتاج حلول برمجية فعلية تلبي متطلبات المستخدمين وتُستخدم في البيئات الواقعية.

في ضوء فهم العلاقة التكاملية بين **هندسة البرمجيات وإنتاجها**، تبرز الحاجة إلى دراسة المسار العملي الذي تمر به البرمجيات منذ نشوء الفكرة الأولية وحتى تسليم المنتج البرمجي النهائي.

يُعرف هذا المسار بـ **"دورة حياة تطوير البرمجيات"** (**Software Development Life Cycle – SDLC**)، وهو إطار منهجي يتألف من مجموعة من المراحل المتسلسلة التي تهدف إلى ضمان إنتاج نظام برمجي فعّال، عالي الجودة، ويلبي متطلبات المستخدمين من الناحية الوظيفية والتقنية على حد سواء. يُعتمد هذا النموذج كأداة تنظيمية وإدارية تُمكِّن الفرق البرمجية من تخطيط وتنفيذ ومتابعة عمليات التطوير بشكل منظم وقابل للقياس.

في ضوء فهم العلاقة التكاملية بين **هندسة البرمجيات وإنتاجها**، تبرز الحاجة إلى دراسة المسار العملي الذي تمر به البرمجيات منذ نشوء الفكرة الأولية وحتى تسليم المنتج البرمجي النهائي.

يُعرف هذا المسار بـ **"دورة حياة تطوير البرمجيات"** (**Software Development Life Cycle – SDLC**)، وهو إطار منهجي يتألف من مجموعة من المراحل المتسلسلة التي تهدف إلى ضمان إنتاج نظام برمجي فعّال، عالي الجودة، ويلبي متطلبات المستخدمين من الناحية الوظيفية والتقنية على حد سواء. يُعتمد هذا النموذج كأداة تنظيمية وإدارية تُمكِّن الفرق البرمجية من تخطيط وتنفيذ ومتابعة عمليات التطوير بشكل منظم وقابل للقياس.

دورة حياة البرمجيات

مرحلة التخطيط للنظام

تُعدّ مرحلة التخطيط حجر الأساس في دورة حياة تطوير البرمجيات، إذ تهدف إلى تحديد مبررات بناء النظام، ومدى جدواه، ومتطلبات نجاحه. وتتضمن هذه المرحلة الإجابة على أسئلة جوهرية مثل:

1. لماذا نحتاج هذا النظام؟
2. ما القيمة التي سيقدمها؟
3. هل يمكننا تطويره ضمن الموارد المتاحة؟
4. وما المدة الزمنية والتكلفة المطلوبة؟

تنقسم هذه المرحلة إلى محورين أساسيين :

1.تهيئة المشروع و تشمل :

تحديد أهمية المشروع ، دراسة الجدوى (الفنية ، الاقتصادية ، القانونية) ، تحليل المخاطر.

2.إدراة المشروع و تشمل تحديد :

حجم الفريق و أدواره ، المهارات و التقنيات المستخدمة ، الجدول الزمني .

مخرجات هذه المرحلة :

1. خطة المشروع .
2. تقرير الجدوى و تحليل المخاطر .

مرحلة تحديد متطلبات النظام

تُعدّ هذه المرحلة من أكثر مراحل تطوير البرمجيات أهمية، حيث تُبنى عليها كافة مراحل التصميم والتنفيذ و يُركّز فيها على تحديد "ما الذي يجب أن يفعله النظام؟ " و " ما هي خصائصه؟ "، من خلال التواصل المكثّف مع أصحاب المصلحة والمستخدمين.

أنواع المتطلبات :

1. المتطلبات الوظيفية (Functional Requirements) :

يقصد بها الوظائف التي يجي أن يُنفذها النظام .

2. المتطلبات الغير وظيفية (Non - Functional Requirements) :

يقصد بها الخصائص العامة للنظام و التي لا تتعلق بوظائفه مباشرة و تُصنف ضمن مفهوم عوامل الجودة للنظام مثل قابلية الاستخدام و قابلية الصيانة و الأداء .. الخ.

تصنيفات أخرى للمتطلبات:

1. **متطلبات العميل :** رؤية العميل و الهدف العام .
2. **متطلبات المستخدم :** سلوك النظام من وجهة نظر المستخدم النهائي .
3. **متطلبات النظام :** التفاصيل التقنية التي تحدد كيفية بناء النظام .

مخرجات هذه المرحلة :

وثيقة (SRS – Software Requirements Specification):

تشمل جميع المتطلبات الوظيفية وغير الوظيفية، وتُعد مرجعًا أساسيًا في مراحل التصميم والتطوير

مرحلة تحليل متطلبات النظام

بعد جمع وتحديد متطلبات النظام، يتم الانتقال إلى مرحلة تحليل النظام وبناء النماذج التحليلية. تهدف هذه المرحلة إلى ترجمة المتطلبات إلى نماذج مفهومة تمثّل سلوك النظام وهيكله، وتساعد في الكشف عن الغموض أو التعارض في المتطلبات بمعنى آخر كما درسنا سابقاً هي استخدام مفهوم النمذجة في النظام .

تُستخدم عدة أنواع من النماذج :

1. النماذج الوظيفية :

مثل مخطط حالة الاستخدام و مخطط النشاط و ذلك لتوضيح تسلسل العمليات و تفاعل المستخدمين مع النظام .

2. النماذج الهيكلية :

مثل مخطط الفئات و ذلك لتمثيل مكوّنات النظام على شكل كائنات تحتوي على خصائص ووظائف.

3. النماذج السلوكية :

مثل مخطط التتابع و مخطط التوقيت و التي توضّح كيفية تفاعل الكائنات داخليًا لتنفيذ العمليات.

مخرجات هذه المرحلة :

وثيقة اقتراح النظام (system proposal):

تتضمن تحليلًا شاملًا للوظائف، والنماذج التحليلية، ومخططات النمذجة ، وتُقدّم إلى لجنة القبول لاتخاذ قرار البدء بالتصميم أو إعادة تحليل المتطلبات

مرحلة تصميم النظام

هي المرحلة التي نحدّد فيها **كيفية بناء النظام** من الناحية التقنية، بناءً على المتطلبات المُحللة سابقًا. يتم فيها اتخاذ قرارات تفصيلية حول معمارية النظام، تصميم قواعد البيانات، الواجهات، والمكونات البرمجية.

الخطوات الأساسية للتصميم :

1. وضع استراتيجية التصميم .
2. اختيار معمارية التصميم مثل clean architecture , mvc architecture.
3. تحديد و تصميم مواصفات قواعد البيانات .
4. تصميم الواجهات للنظام أي تحديد كيفية شكل الواجهات باستخدام أدوات مثل فيقما .
5. تصميم المعمارية التقنية أي تحديد الأجهزة المطلوبة و اختيار منصات التطوير .. الخ.

مخرجات هذه المرحلة :

1. مستند تصميم معمارية النظام .
2. مستند تصميم واجهات المستخدم .
3. مستند تصميم قواعد البيانات .
4. مستند البرامج و المنصات المستخدمة .

مرحلة تطوير النظام

هي المرحلة التي يتم فيها **بناء النظام فعليًا** من خلال برمجة الوظائف المحددة وربطها معًا، بالاعتماد الكامل على وثائق التحليل والتصميم.

يتم في هذه المرحلة تحويل التصميم إلى كود باستخدام لغات البرمجة المناسبة ، و تشمل :

1. برمجة الواجهة الامامية **(Front-end)**.
2. برمجة الواجهة الخلفية و قواعد البيانات **(Back-end)**.
3. دمج المكونات و ربطها .

مخرجات هذه المرحلة :

1. الكود البرمجي الكامل للنظام .
2. وثيقة الإجراءات البرمجية .
3. دليل الإعداد و التشغيل .

مرحلة اختبار النظام

هدفها التحقق من أن النظام يعمل وفق المتطلبات المحددة وكشف العيوب قبل التسليم.

خطوات و أنواع الاختبارات :

1. اختبار الوحدة :

ويجري على دالة أو صنف أو جزء محدد من وحدة برمجية للتأكد من أن الدخل الصحيح يعطي الخرج المطلوب.

2. اختبار التكامل :

يختبر التفاعل المتبادل بين مجموعة محددة من الأصناف لإنجاز وظيفة محددة.

3. اختبار النظام :

يختبر تفاعل جميع الأصناف مع بعضها لإعطاء وظيفة متكاملة دون أخطاء، وهو مشابه لاختبار التكامل لكن على صعيد أوسع.

.

4. اختبار القبول :

يتم من قِبل المستخدمين للتأكد من مطابقة النظام لاحتياجاتهم.

مرحلة نشر النظام و صيانته

تبدأ مرحلة النشر بعد الانتهاء من تطوير واختبار النظام، حيث يُنشر النظام في بيئة العمل الحقيقية ويُسلَّم للمستخدمين النهائيين ، تهدف إلى ضمان جاهزية النظام للاستخدام واستمراريته بكفاءة بعد الإطلاق.

أما مرحلة الصيانة تبدأ بعد النشر وتستمر طوال فترة تشغيل النظام، وتشمل:

1. تصحيح الأخطاء البرمجية التي تظهر بعد الاستخدام.
2. إجراء تحسينات وتحديثات على النظام حسب التغذية الراجعة من المستخدمين.
3. التكيّف مع تغيرات البيئة التقنية أو متطلبات العمل.

مع التنبيه على ضرورة التوثيق في مرحلة الصيانة .

منهجيات تطوير البرمجيات

منهجية waterfall

هي منهجية خطية تقسم المشروع إلى مراحل متسلسلة، بحيث لا تبدأ مرحلة إلا بعد انتهاء المرحلة السابقة، وتُستخدم في المشاريع التي تتطلب تخطيطًا دقيقًا وتنفيذًا مرحليًا واضحًا.

المزايا :

1. التخطيط المسبق الدقيق.
2. وضوح تسلسل المهام.
3. سهولة تتبع التقدم.
4. توثيق كامل لكل مرحلة.

العيوب :

1. صعوبة التراجع عند اكتشاف أخطاء.
2. تأخير اكتشاف المشاكل حتى المراحل المتأخرة.
3. غير مرنة مع التغييرات المتكررة.
4. أي تأخير في مرحلة يؤثر على كامل الجدول الزمني.

مناسبة ل :

1. المشاريع ذات الأهداف المحددة والواضحة.
2. المشاريع التي لا تعاني من قيود زمنية.
3. العمليات المتكررة القابلة للتوثيق والتوحيد.

منهجية Agile

منهجية الأجايل هي طريقة مرنة في مجال تطوير البرمجيات، تعتمد على تقسيم المشروع إلى أجزاء صغيرة تُنفذ في فترات زمنية قصيرة تسمى "("Sprints") ، مع التفاعل المستمر مع العميل لتحديث المتطلبات وتطوير المنتج تدريجيًا حتى الوصول إلى النسخة النهائية.

المزايا :

1. مرونة عالية مع تغير المتطلبات.
2. تحسين مستمر للمنتج.
3. تسليم سريع و متدرج.
4. تعزيز التواصل بين الفريق و العميل.
5. تقليل المخاطر لأن التقييم بشكل دوري .

مناسبة ل :

1. المشاريع التي تتغير متطلباتها بشكل سريع.
2. المشاريع التي تتطلب التحسين بشكل مستمر .

العيوب :

1. صعبة في المشاريع التي تتطلب تخطيط دقيق من البداية.
2. استهلاك الوقت بسبب كثرة الاجتماعات.
3. يحتاج إلى فريق ذو خبرة و مهارة عالية .
4. صعوبة تحديد الجدول الزمني بشكل دقيق.
5. قلة التوثيق .

منهجية scrum Agile

Scrum هو إطار عمل قائم على Agile، يُقسِّم المشروع إلى فترات زمنية قصيرة تُسمى Sprints (عادةً من أسبوعين إلى أربعة أسابيع)، وفيه يقوم الفريق بتنفيذ مجموعة من المهام المحددة مسبقًا.

مكونات ال **scrum** :

الفريق :

1. **Product Owner** : يحدد متطلبات المشروع و ألوياتها .
2. **Scrum Master** : يسهل العملية و يقوم بحل العوائق .
3. **Development Team** : ينفذ المهام البرمجية داخل ال **scrum**.

الاجتماعات الأساسية :

1. **Sprint planning** : لتخطيط المهام قبل بدء **Sprint** .
2. **Scrum Daily** : اجتماع يومي لمتابعة التقدم .
3. **Sprint review** : استعراض ما تم إنجازه في نهاية **Sprint** .

المنتجات الأساسية :

1. **product backlog** : قائمة بجميع متطلبات المشروع .
2. **Sprint backlog** : المهام المختارة للتنفيذ خلال **Sprint** .
3. **increment** : الجزء المنجز من المشروع بعد كل **Sprint** .

منهجية Kanban Agile

kanban هي طريقة لتنظيم وإدارة العمل تُستخدم لتحسين تدفق المهام وزيادة الكفاءة. تعتمد على مبدأ الشفافية البصرية في تتبع تقدم الأعمال.

مفاهيم ال kanban :

1. **Kanban board : تقسم إلى مراحل تمثل سير العمل مثل :**

TO DO : المهام المطلوب تنفيذها

In progress : المهام التي يتم العمل عليها

Done : المهام المنجزة

مع ملاحظة أن كل بطاقة تمثل مهمة واحدة و تنتقل من عمود لأخر حسب التقدم في تنفيذها .

أيضاً يجب تحديد عدد معين من المهام التي يجب تنفيذها في كل مرة لتجنب التشتت .

المزايا :

1. **يوفّر تصورًا بصريًا لجميع مراحل تنفيذ المهام، مما يسهل المتابعة.**
2. **يساعد على اكتشاف المراحل التي تتراكم فيها المهام ومعالجتها بسرعة.**
3. **يمكن تعديل المهام وإعادة ترتيب الأولويات بسهولة دون التأثير على النظام العام.**
4. **باستخدام حدود للمهام الجارية ، يحسن من التركيز للفريق و يقلل من التشتت .**

Kanban Vs Scrum

Kanban

1. لا توجد أدوار واضحة لكل عضو في الفريق .
2. لا يتبع منهجية تكرارية .
3. لا يقسم المشكلات إلى مهام صغيرة .
4. أقل صرامة من Scrum
5. يعتمد على تصور بصري لتنفيذ المهام .
6. يستخدم بطاقات مهام لمتابعة التقدم .
7. مناسب عند قلة الموارد و المهام .
8. أي عضو يمكنه حل المشكلات .
9. يتأثر تدفق العمل عند غياب عضو.
10. التقدير الزمني أقل أهمية.
11. مناسب للمشاريع ذات الأولويات الثابتة .
12. من أدوات هذه المنهجية :

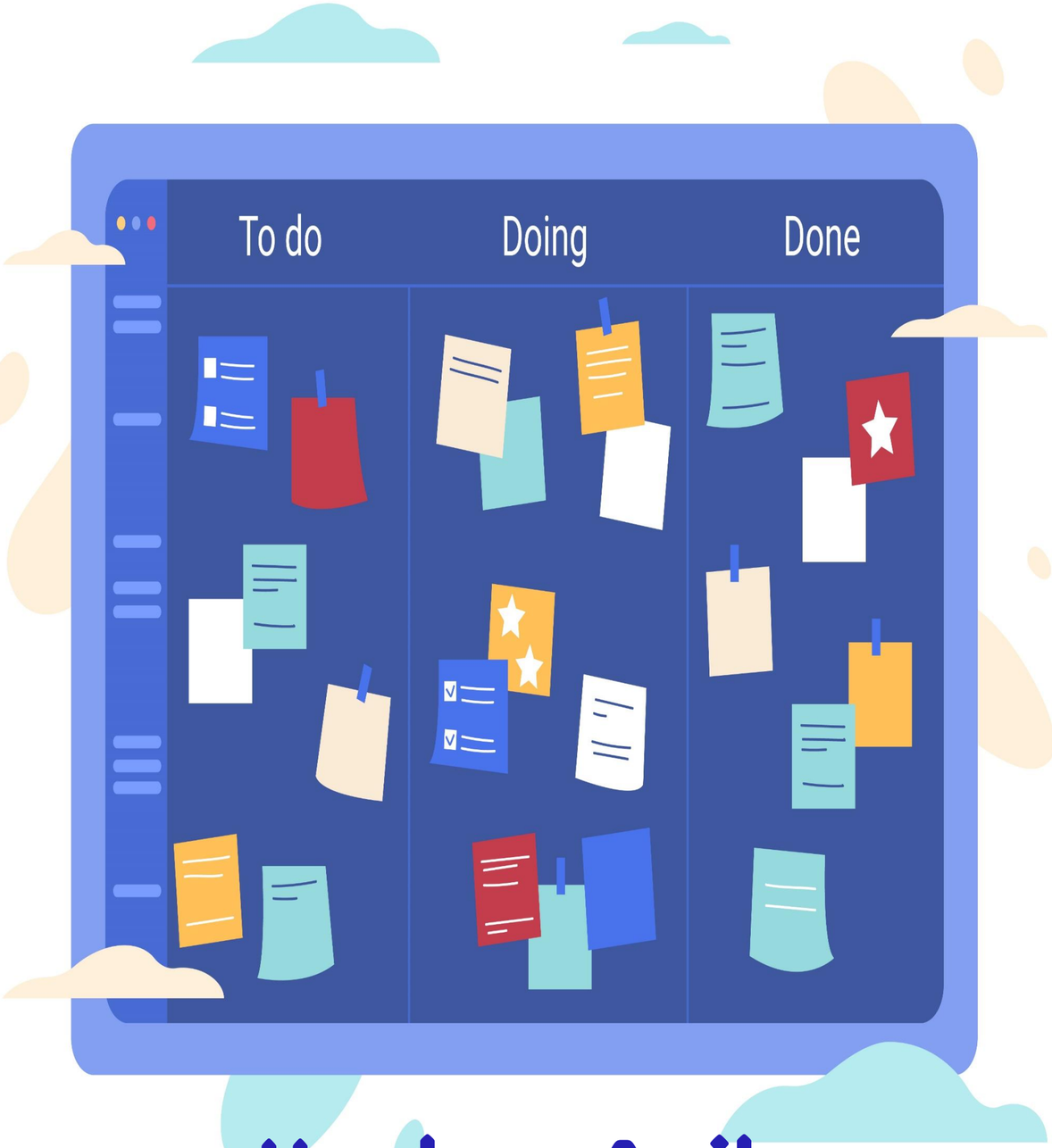
Jira Software , Anasa ,swiftKanban and more.

أطر منهجية Agile

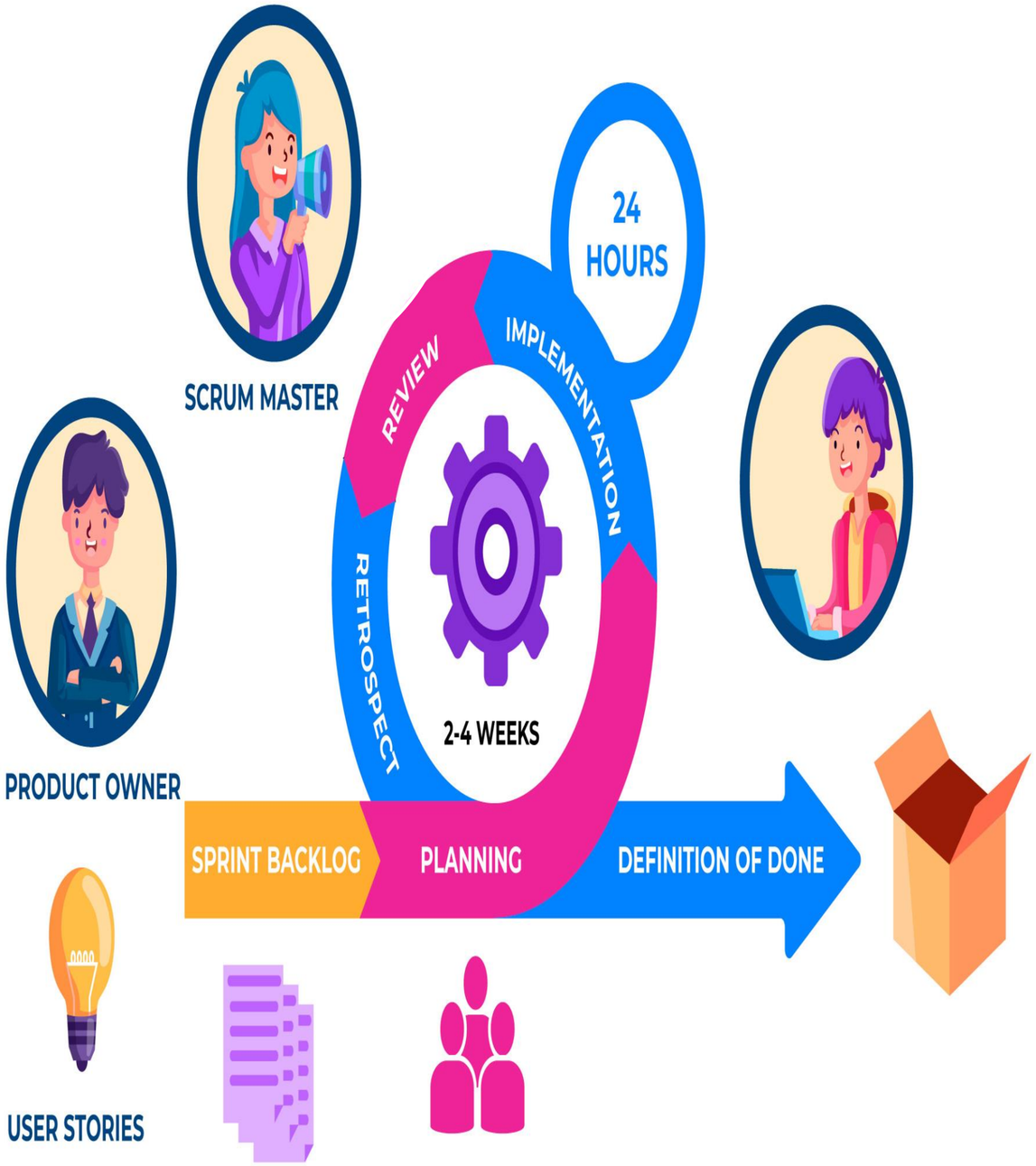
Scrum

1. يحدد أدوار واضحة لكل عضو في الفريق .
2. يتبع منهجية تكرارية .
3. يقسم المشكلات إلى مهام صغيرة لحلها .
4. منهجية صارمة و موضحة بالتفصيل .
5. لا يعتمد على تصور بصري لتنفيذ المهام .
6. يستخدم sprint لمتابعة التقدم .
7. غير مفضل عند قلة الموارد .
8. يحل Scrum master المشكلات .
9. لا يتأثر التقدم بخروج أحد الأعضاء .
10. التقدير الزمني ضروري للتخطيط .
11. مناسب للمشاريع ذات الأولويات المتغيرة .
12. من أدوات هذه المنهجية :

Jira Software , Axosoft ,VivifyScrum and more.



Kanban Agile



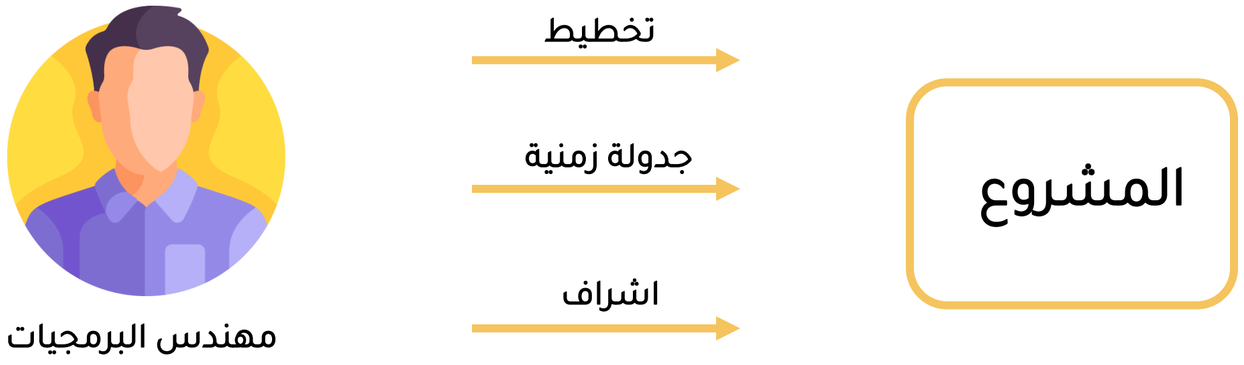
scrum Agile

إدارة المشاريع البرمجية:

يمكن وصف المشاريع البرمجية بأنها أنشطة تصميم وتنظيم تطوير البرمجيات وفقًا لعدة عوامل أبرزها الوقت والتكلفة والجودة والنطاق الواسع، وهي فئة فرعية من عملية تطوير البرمجيات.

وبالتالي تُعرف إدارة المشاريع البرمجية بأنها عمليات وممارسات ضبط لتخطيط مشاريع البرمجيات وجدولتها والإشراف عليها، إذ تعمل على تخصيص الموارد لتطوير برامج الكمبيوتر التي تلبي المتطلبات المُحددة سلفًا، وفي هذا البرنامج لا بد من وضوح طول المشروع وفترته وتكلفته بالنسبة للعميل والمطورين.

وفي نظام إدارة المشاريع البرمجية يراقب مديرو المشاريع وأصحاب المصلحة والمستخدمين التكاليف، ويديرون الميزانية والجودة، ويستطيعون استخدام هذا النظام كنظام إداري.



في الحقيقة يقوم مدراء المشاريع البرمجية بنفس العمل الذي يقوم به مدراء المشاريع العمل الهندسية الأخرى غير أن مهمتهم تختلف عن مهمة بقية المدراء في النقاط التالية :

1. أن المنتج الذي يشرفون على تطويره غير محسوس.
2. إن الطبيعة المرنة و المطواعة للبرمجيات توحي بسهولة التعديل في اية لحظة ليتم استدراك أمر سبق نسيانه ، غير أن هذا الامر يقود الى اجراء تعديلات كثيرة مما يجعل من الصعب مراقبة تطور سير العمل.
3. لا توجد معايير معترف بها عالميا للإجراءات البرمجية .
4. أن كل مشروع برمجي ضخم يبدو فريداً من نوعه لا يتكرر .

أنشطة إدارة المشروع

يصعب توصيف مهام مدير المشروع البرمجي ، لان هذا التوصيف معرض للتغيير و التبديل في بعض أجزائه و ذلك حسب طبيعة النتج الذي يجري تطويره و المؤسسة التي تقوم بالتطوير ، غير أننا سنذكر جميع المهام التي يمكن للمهام ان يقوم ببعضها أو بها جميعا و هي كالتالي :

أنشطة إدارة المشروع

1. كتابة مقترح المشروع

و هي المهمة الأولى التي يقوم بها المدير عندما يقدم عرضاً بهدف الفوز بعقد لتطوير نظام برمجي .

أنشطة إدارة المشروع

2. التخطيط و وضع جدول زمني للمشروع

يحدد المدير الأنشطة التي يجب القيام بها و النواتج التي سيقدمها المشروع ، كما يضع خطة لمتابعة تنفيذ المشروع أي جدول زمني و الشائع في مجالنا هو استخدام مخطط غانط للجدول الزمني.

أنشطة إدارة المشروع

3. تقدير تكلفة المشروع :

حيث يقدر المدير الكُلف و الموارد اللازمة لتنفيذ خطة المشروع .

أنشطة إدارة المشروع

4. مراقبة العمل في المشروع و إجراء المراجعات

و هي من المهام المستمرة في إدارة المشاريع حيث يقوم المدير بمقارنة الهطة الموضوعة و التكلفة المتوقعة مع واقع تطور سير العمل و المبالغ المصروفة ، كما يقوم بإجراء عدداً من المراجعات الرسمية و ذلك لمراجعة تطور سير العمل و التأكد من ان المشروع الجاري العمل به يتفق مع المطلوب .

أنشطة إدارة المشروع

5. اختيار العاملين في المشروع و تقييمهم

يقوم المدراء باختيار فريق العمل ، و يجب أن يدرك المدير أنه قد لا يتمكن من توظيف الأشخاص المناسبين و ذوي الخبرة للعمل في المشروع للأسباب الاتية :

1. لا تتحمل ميزانية المشروع الأجور العالية .
2. لا يوفر فريق بالخبرة المطلوبة .
3. رغبة شركة التطوير بتدريب فريقها الخاص و تكوين مهارات جديدة من خلال هذا المشروع

أنشطة إدارة المشروع

6. كتابة التقارير و إجراء العروض

يعتبر المدير مسؤولا عن كتابة التقارير و اجراء العروض التي تبين تقدم العمل للزبون .

تحديات إدارة المشاريع البرمجية

1. توقعات غير واضحة و غير محددة

من العوامل التي تعوق تقدم المشروع، فشل المديرون في تحديد أهدافه وتوقعاته من العملاء، وهو ما يؤدي إلى انحراف النتائج عن المطلوبة، وبالتالي تتأثر سُمعة ومكانة المنظمة في السوق بالسلب.
ومن عواقب غموض الأهداف والتوقعات، زيادة التكلفة دون تحقيق الجودة المطلوبة، وهو ما يثبط من حافز الفريق بالكامل.

الحل :

يتمثل حل هذه المشكلة في قيام المديرون برصد احتياجات وتوقعات العملاء باستخدام أدوات مختلفة في إدارة المشاريع مثل Slack و Asana و Microsoft Teams، حتى يستطيع الفريق فهم ما ينبغي القيام به وآلية ذلك.

تحديات إدارة المشاريع البرمجية

2. ضيق الوقت

تواجه فرق إدارة مشاريع البرمجيات مشكلة ضيق الوقت المُحدد للانتهاء من المشروع، وهو ما يؤثر بالسلب على جودة البرنامج، كما يصعب على فريق التطوير ومديري المشاريع التعامل مع المواعيد النهائية الضيقة نتيجة التغيرات التكنولوجية والمنافسات العالمية.

الحل :

يجب التواصل مع العملاء لتحديد المواعيد النهائية للمشاريع.

تحديات إدارة المشاريع البرمجية

3. تغير احتياجات المشاريع

خلال تنفيذ المشروع، قد تحدث تغييرات في احتياجات العملاء، وبالتالي يضطر المديرون لزيادة الوقت والموارد، وهو ما يؤثر على المشروع بالسلب.

الحل :

يكمن حل هذه المشكلة في استخدام المنهجية المرنة، حتى يستطيع الفريق إدارة المشروع بكفاءة في حال تغير المتطلبات.

تحديات إدارة المشاريع البرمجية

4. ضعف التواصل

يتأثر المشروع بالسلب عندما يضعف التواصل مع العملاء وأصحاب المصلحة الآخرين وفريق التطوير، لأن هذه المشكلة ستؤدي إلى تداخل الأدوار والمسؤوليات وبالتالي نشوب الصراعات بين الأعضاء.

الحل :

لا بد من تواصل مديرو المشاريع مع العملاء وكل عضو في الفريق، لضمان نجاح المشروع.

تحديات إدارة المشاريع البرمجية

5. إدارة المهارات

عدم مهارة وكفاءة أعضاء الفريق بما فيه الكفاية، يزيد من احتمالية فشل المشروع، ويضع مدير المشروع في موقف بالغ الصعوبة.

الحل :

قبل بدء المشروع، يتعين على المديرين تقييم مهارات وخبرات أعضاء الفريق، ومعرفة مدى حاجتهم إلى تدريب فريق التطوير.

تحديات إدارة المشاريع البرمجية

6. تغير التقنيات

تشهد التقنيات تغييرًا سريعًا وبصورة مستمرة، فتقنية اليوم قد لا تجدي نفعًا غدًا، وذلك لأن صناعة تكنولوجيا المعلومات هي واحدة من الصناعات الأسرع نموًا.

الحل :

لا بد من مواكبة مديرو المشاريع أحدث التقنيات التكنولوجية، لزيادة سرعة وكفاءة عملية التطوير.

تحديات إدارة المشاريع البرمجية

7. إحباط أعضاء الفريق

يشعر أعضاء الفريق بالإحباط عندما يواجهون توقعات غير واقعية ومواعيد نهائية ضيقة، وهو ما يؤثر على أدائهم بالسلب.

الحل :

يتعين على المديرين وضع توقعات واقعية بالتحدث مع العملاء، إضافة إلى تحديد مواعيد نهائية مناسبة، مع التأكد من استعداد كل عضو في الفريق للمشروع.

تحديات إدارة المشاريع البرمجية

8. إلغاء المشروع

من أسوأ التحديات التي تواجه إدارة مشاريع البرمجيات إلغاء المشروع في منتصفه لوجود أسباب مختلفة.

الحل :

تستطيع الشركة تفادي هذه المشكلة من البداية باختيار المشروع بحكمة قبل توقيع العقد.

تحديات إدارة المشاريع البرمجية

9. عدم التقدير

يؤدي عدم تقدير مهارات وخبرات أعضاء فريق المشروع إلى تأثر أدائهم بالسلب، وبالتالي يتأثر المشروع ككل.

الحل :

لا بد من تقدير مهارات وخبرات أعضاء الفريق من قِبل المديرون، مع اعتراف أصحاب المصلحة بها.

تحديات إدارة المشاريع البرمجية

10. عدم إدارة المخاطر

عدم تنبؤ مديرو المشاريع بالمخاطر المُحتملة يؤثر بالسلب على سير المشروع، خاصة عندما تظهر مخاطر مثل نقص الميزانية أو الوقت أو الموارد.

الحل :

يتعين على مديري المشاريع التنبؤ بالمخاطر المُحتمل وقوعها خلال تنفيذ المشروع، ووضع تدابير مختلفة لها.

أدوات إنتاج البرمجيات

أدوات إدارة المشاريع

تُستخدم هذه الأدوات لتخطيط المشروع، تنظيم المهام، متابعة سير العمل، وضمان الالتزام بالمواعيد النهائية ، من هذه الأدوات :

Jira , Trello

أدوات التعاون

تدعم هذه الأدوات العمل الجماعي، وتُسهم في إدارة التعديلات على الشيفرة البرمجية بكفاءة ، و من هذه الأدوات :

Git , GitHub

أدوات التحليل

تُستخدم في المراحل المبكرة لفهم احتياجات النظام ورسم تصور دقيق لآلية عمله، و من هذه الأدوات :

Draw.io

أدوات التصميم

تُستخدم لتصميم واجهات المستخدم وتقديم نماذج أولية تفاعلية Prototypes، و من هذه الأدوات :

Figma , Adobe Xd

أدوات التطوير

هي بيئات متكاملة لكتابة الشيفرة البرمجية وتصحيح الأخطاء ، أشهرها :

VS Code

البرمجيات مفتوحة المصدر

البرمجيات مفتوحة المصدر هي برمجيات يتم توفير شيفرتها المصدرية بشكل علني ومجاني، مما يتيح للمستخدمين والمطورين الاطلاع عليها، تعديلها، وتوزيعها بحرية وفقًا لشروط رخصة مفتوحة المصدر.

تقوم هذه البرمجيات على مبدأ الشفافية والتعاون المجتمعي، حيث تُمكن المجتمعات التقنية من المساهمة في تطوير البرمجيات وتحسينها بشكل مستمر.

فوائد البرمجيات مفتوحة المصدر

1. التعاون و الابتكار

تتيح البرمجيات مفتوحة المصدر للمطورين من مختلف أنحاء العالم العمل معًا في مشروع واحد. هذا التعاون يُسرّع من عملية التطوير ويؤدي إلى حلول مبتكرة وأكثر موثوقية، لأن عددًا كبيرًا من العقول يشارك في تحسين المشروع.

2.المرونة و التخصيص

من أبرز مزايا المصادر المفتوحة هي إمكانية تعديل الكود حسب الحاجة. هذا يعطي المطورين والمؤسسات حرية تخصيص البرنامج ليتماشى مع متطلباتهم الخاصة، مما يؤدي إلى أداء أفضل وحلول أكثر دقة.

3. تقليل التكاليف

غالبًا ما تكون هذه البرمجيات مجانية أو منخفضة التكلفة، مما يساعد المؤسسات على تقليل المصاريف المرتبطة بتراخيص البرامج التجارية. يمكن توجيه هذه الموارد إلى جوانب أخرى مثل تطوير البنية التحتية أو تحسين الخدمات.

فوائد البرمجيات مفتوحة المصدر

4. التوافق مع المعايير

غالبًا ما تلتزم البرمجيات مفتوحة المصدر بالمعايير التقنية المفتوحة، مما يسهل عملية دمجها مع أنظمة أخرى. هذا التوافق يُقلل من مشاكل التوافق ويساعد المؤسسات على بناء أنظمة مرنة.

5. الاستمرارية

حتى إذا توقفت جهة معينة عن تطوير المشروع، يمكن لأي جهة أخرى من المجتمع أن تستلم المشروع وتواصل العمل عليه. هذا يضمن استمرارية البرنامج وعدم الاعتماد على شركة واحدة فقط.

6. دعم المجتمع

كل مشروع مفتوح المصدر تقريبًا تحيط به مجموعة من المطورين الذين يقدمون المساعدة، سواء من خلال منتديات، أو توثيق، أو شروحات. هذا الدعم يُسهّل على المستخدمين والمطورين الجدد تعلم البرنامج والتعامل معه.

سلبيات البرمجيات مفتوحة المصدر

1. **واجهة المستخدم الغير صديقة :** العديد من هذه البرمجيات تُصمّم بالتركيز على الأداء والمرونة، وليس على سهولة الاستخدام، مما يجعلها صعبة على المستخدمين غير التقنيين، خصوصًا عند المقارنة بالبرمجيات التجارية.

2. **غياب التوثيق الرسمي :** رغم وجود مجتمعات داعمة، إلا أن كثيرًا من المشاريع تفتقر إلى توثيق رسمي شامل ومحدث، مما يصعّب على المستخدمين الجدد فهم النظام أو تعديله.

3. **الدعم الفني المحدود :** حيث توفر معظم البرمجيات مفتوحة المصدر دعمًا رسميًا مباشرًا، مما يجعل المستخدمين يعتمدون على المنتديات والمجتمعات، والتي قد لا تكون دائمًا سريعة أو فعالة في تقديم الحلول للمشاكل المعقدة.

4. صعوبة التثبيت و التكوين : تتطلب بعض البرمجيات مفتوحة المصدر عمليات إعداد معقدة، تحتاج إلى خبرة فنية عالية، مما قد يشكل عائقًا أمام استخدامها من قبل غير المختصين.

أشهر البرمجيات مفتوحة المصدر هو نظام التشغيل LINUX

هو نظام تشغيل مفتوح المصدر يستخدم على نطاق واسع في مختلف المجالات، مثل الخوادم، أجهزة الكمبيوتر، الهواتف الذكية، وحتى الأجهزة المدمجة. تم تطويره في عام 1991 بواسطة **لينوس تورفالدس**، ويتميز بمرونته واستقراره، مما جعله الخيار الأول للمطورين والشركات التقنية.

مميزات LINUX

1. مفتوح المصدر: الكود المصدري متاح للجميع، مما يسمح للمطورين بتعديله وتحسينه حسب احتياجاتهم.

2. التكلفة المنخفضة: على عكس أنظمة التشغيل المغلقة مثل Windows، يُعد Linuxمجانيًا بشكل أساسي، مما يقلل من تكاليف الاستخدام.

3. الأمان العالي: يعتبر Linuxمن أكثر أنظمة التشغيل أمانًا بفضل بنيته المصممة لمقاومة البرمجيات الضارة والهجمات الإلكترونية.

4. الأداء والاستقرار: يتميز بالقدرة على العمل بكفاءة حتى على الأجهزة القديمة وبأداء مستقر لفترات طويلة دون الحاجة إلى إعادة التشغيل

5. مجتمع نشط: يتمتع Linuxبمجتمع كبير من المطورين والمستخدمين الذين يساهمون في تحسينه ودعمه باستمرار.

أشهر توزيعات LINUX

1. Ubuntu : صديق للمبتدئين ويستخدم في أجهزة الكمبيوتر المكتبية والخوادم.

2. Debian : معروف باستقراره العالي ومناسب للخوادم.

3. Kali Linux : متخصص في اختبارات الأمان والاختراق.

أهداف إنتاج البرمجيات

- **تلبية احتياجات المستخدم** و ذلك عن طريق تصميم برمجيات تتمحور حول المستخدم، تلبي متطلباته وتوفّر تجربة استخدام فعّالة وسلسة .
- **الوظائف و الاعتمادية** و ذلك عن طريق تقديم وظائف دقيقة وخالية من الأخطاء تضمن أداء المهمات المطلوبة بثبات وموثوقية.
- **التوسعية** و ذلك عن طريق تصميم النظام ليستوعب نمو الأعمال دون التأثير على الأداء أو الاستقرار.
- **الأمان** و ذلك عن طريق تضمين آليات حماية قوية ضد التهديدات السيبرانية وضمان خصوصية البيانات..
- **سهولة الصيانة** و ذلك عن طريق كتابة شيفرة منظمة وقابلة للتعديل تسهّل التحديثات المستقبلية وتطوير الميزات الجديدة.
- **الكفاءة من حيث التكلفة بحيث يتم** تحقيق أقصى قيمة بأقل تكلفة ممكنة، مع الحفاظ على جودة المنتج ومخرجاته.
- **الالتزام بالمواعيد** و ذلك عن طريق إدارة الوقت بفعالية لتسليم المنتج ضمن الجدول الزمني المحدد، وتقليل المخاطر.
- **ضمان الجودة** و ذلك من خلال **استخدا**م تقنيات اختبار ومراجعة صارمة للتأكد من مطابقة البرمجيات للمعايير المطلوبة.
- **التوثيق** و ذلك عن طريق إعداد وثائق شاملة توضح آلية عمل النظام، مما يُسهل الصيانة والاستخدام.

تحديات إنتاج البرمجيات

1. **الديون التقنية** و ذلك عن طريق تراكم الحلول السريعة الذي يؤدي إلى شيفرة ضعيفة، ما يصعّب التوسعة لاحقًا ويؤثر على الأداء

2. **تعقيد التكامل من خلال** دمج أنظمة أو خدمات مختلفة بحيث يسبب مشاكل توافق وتبادل بيانات، ويحتاج لتخطيط دقيق.

3. **تغير المتطلبات** يقصد بها أن المتطلبات غير الثابتة تُسبب "زحف النطاق"، مما يؤدي لتجاوز الميزانية والتأخير في التسليم.

4. **قيود الموارد** أي الوقت، الميزانية، وعدد المطورين تؤثر مباشرة في سرعة الإنجاز واتساع المميزات.

5. **المخاطر الأمنية** أي ضعف التشفير أو المصادقة يعرّض الأنظمة للاختراق .

ضوابط الأمان في إنتاج البرمجيات

تعد ضوابط الأمان ضرورية لضمان إنتاج البرمجيات بشكل آمن وحمايتها من التهديدات المحتملة طوال دورة حياتها يجب تحديد وتطبيق هذه الضوابط في بيئة تطوير البرمجيات للتخفيف من المخاطر وضمان الأمان القوي .

متطلبات الأمان في إنتاج البرمجيات

تعد متطلبات الأمان أساس عملية تطوير البرمجيات الآمنة حيث تضمن أن البرمجيات تلبي المعايير الضرورية للسرية والسلامة والتوافر وهو أمر أساسي لحماية البيانات وضمان موثوقية النظام.

السرية

Confidentiality

تضمن السرية عدم الكشف عن المعلومات الحساسة للأفراد أو الأنظمة غير المصرح لهم عادة ما يتم تطبيقها من خلال التشفير وضوابط الوصول وطرق المصادقة الآمنة

السلامة

Integrity

تضمن السلامة دقة البيانات وعدم العبث بها يتم الحفاظ عليها من خلال التحقق من التحقق من صحة البيانات والتواقيع الرقمية وآليات التحقق الأخرى التي تكتشف التعديلات غير المصرح بها

التوافر

Availability

يضمن التوافر أن تكون الأنظمة والبيانات متاحة عند الحاجة يشمل ذلك تنفيذ الحلول الاحتياطية وخطط استعادة الكوارث لمنع التوقف وفقدان البيانات

التوثيق في هندسة البرمجيات و انتاجها

يُعد توثيق هندسة البرمجيات عنصرًا أساسيًا في كل مشروع ناجح لإنتاج البرمجيات، حيث يُساهم في توصيل بنية النظام وتصميمه لجميع المعنيين بالمشروع. كما يُوفّر إطارًا للتواصل الفعّال بين مختلف أصحاب المصلحة، مثل المهندسين، ومديري المشاريع، والمبرمجين، والعملاء. يؤدي التوثيق الدقيق إلى تعزيز الفهم المشترك للوظائف والأهداف، ويُسهّل الصيانة المستقبلية، ويُحسّن كفاءة سير العمل داخل الفريق.
تتكوّن بنية مشروع البرمجيات من مجموعة القرارات التقنية والتنظيمية التي تحدد كيفية بناء الأنظمة والمكونات، والتقنيات المعتمدة، وآليات تفاعل الوحدات، ومسار تطور النظام عبر الزمن. ومن خلال توثيق هذه الخيارات مع مبرراتها، يمكن تحقيق إنتاج برمجيات أكثر استقرارًا، وتقليل مخاطر التعارض وسوء الفهم.

فوائد التوثيق في هندسة البرمجيات و إنتاجها

- **تحسين التواصل حيث** يوفر التوثيق مرجعًا مشتركًا يساعد الفرق الفنية والإدارية على فهم بنية النظام، مما يعزز التنسيق ويقلل من الأخطاء الناتجة عن سوء الفهم.

- **دعم الفهم المنهجي حيث** يمنح التوثيق رؤية شاملة للهندسة الداخلية للبرمجيات، مما يساعد في اتخاذ قرارات مدروسة عند التصميم أو التطوير أو التوسعة.

- **نقل المعرفة حيث** يعد أداة فعالة لتسريع اندماج المطورين الجدد، لاسيما في المشاريع المعقدة أو التي تتعرض لتغيرات دورية في أعضاء الفريق.

- **تحسين الصيانة و التطوير المستقبلي حيث** يمكّن التوثيق الدقيق الفرق من معالجة المشكلات وتطبيق التحديثات دون التأثير على سلامة النظام.

- **الامتثال للمعايير حيث** يُعتبر التوثيق ضرورة في العديد من الصناعات الخاضعة للوائح ومعايير جودة صارمة، مثل الرعاية الصحية والقطاع المالي.

وثيقة هندسة البرمجيات

وثيقة هندسة البرمجيات تُستخدم لتوصيف البنية العامة للنظام البرمجي وتبرير القرارات المعمارية المتخذة. وتُعد مرجعًا أساسيًا للمطورين، ومديري المشاريع، والمختبرين.

تساعد هذه الوثيقة في:

1. توجيه التطوير المستقبلي.
2. تسهيل التواصل بين أعضاء الفريق.
3. توضيح المنطق خلف قرارات التصميم.
4. دعم الصيانة والنقل المعرفي بعد إطلاق النظام.

العناصر الأساسية لوثيقة هندسة البرمجيات

- **سياق النظام و نطاقه حيث** يقدم وصفًا عامًا للنظام من حيث الأهداف، المستخدمين، والبيئة التشغيلية، مما يساعد على تحديد حدود النظام وفهم نطاق عمله.

- **الأهداف و القيود حيث** يتم تحديد المتطلبات الوظيفية وغير الوظيفية التي تؤثر على التصميم المعماري، مثل متطلبات الأمان، الأداء، أو قيود التكنولوجيا المستخدمة.

- **مخططات المكونات يقصد بها** تصورات رسومية مثل UML توضّح المكونات الرئيسية للنظام والعلاقات بينها.

- **مخططات التسلسل** بحيث يتم تمثيل تفاعل المكونات في سيناريوهات محددة (مثل تسجيل الدخول)، وتوضح تدفق الرسائل والتحكم.

- **نماذج البيانات حيث** تعرض هيكل البيانات، الجداول، العلاقات، والقيود، وتدعم تصميم قاعدة البيانات بما يتوافق مع متطلبات النظام.

- **المتطلبات الغير وظيفية حيث** يتم توثيق خصائص الجودة الواجب توفرها، مثل القابلية للتوسع، الأمان، والكفاءة، وتؤثر بشكل مباشر على القرارات المعمارية.

اتجاهات حديثة في إنتاج البرمجيات

الذكاء الاصطناعي و تعلم الالة

أصبح الذكاء الاصطناعي أداة محورية في تطوير البرمجيات، حيث يُستخدم في أتمتة الاختبار، تحليل الكود، وتوليد تعليمات برمجية ذكية. تساهم هذه التقنيات في تحسين الكفاءة وتقليل الأخطاء البشرية.

المنصات المنخفضة أي بدون تعليمات برمجية

تُسهّل هذه المنصات تطوير التطبيقات من خلال واجهات مرئية ومكونات جاهزة، مما يتيح مشاركة غير المبرمجين في العملية ويُسرّع دورة الإنتاج.

DevSecOps

يتكامل الأمان كعنصر أساسي في دورة التطوير، مما يعزز الحماية من التهديدات السيبرانية. يُعد DevSecOps امتدادًا لمبادئ DevOps مع تركيز أكبر على الأمان التلقائي والمبكر.

الحوسبة الطرفية

مع تزايد الأجهزة الذكية، تعتمد الأنظمة الحديثة على المعالجة بالقرب من مصدر البيانات لتقليل التأخير وتحسين الأداء، مما يفرض تحولات معمارية في تصميم البرمجيات.

الحوسبة الكمومية

رغم أنها لا تزال ناشئة، تُعد الحوسبة الكمومية واعدة في مجالات التشفير والمحاكاة. سيتطلب تطورها نماذج برمجة جديدة وتكاملًا مع أنظمة تقليدية.

الواقع الممتد XR

يأخذ الواقع الممتد (AR/VR/MR) دورًا متناميًا في تطوير تطبيقات تعليمية، ترفيهية وتدريبية، مما يتطلب من المطورين مهارات جديدة في تصميم واجهات وتجارب تفاعلية غامرة.

خلاصة إنتاج البرمجيات

يمثل إنتاج البرمجيات عملية متكاملة تبدأ من تحديد المتطلبات وتنتهي بتسليم نظام فعّال وموثوق. تعتمد جودة المنتج النهائي على اتباع منهجيات مناسبة، واستخدام أدوات فعالة، والتوثيق الدقيق، ومراعاة متطلبات المستخدم باستمرار. من خلال تطبيق أفضل الممارسات، يمكن ضمان برمجيات قابلة للصيانة، وقابلة للتوسع، وتلبي تطلعات أصحاب المصلحة.